

Project 2 — Build Guide & Completion Checklist

Amazon Rekognition Image Analysis Pipeline | Ghana KYC Verification System

Instructional Reference · Paired with Project 2 Deliverable Document · Date: April 2026

Project Status: Core Build Complete — Above Requirements

All 4 project phases have been implemented and deployed via Terraform. The group has additionally completed 4 bonus features unprompted. Remaining work covers error handling, CloudWatch alarms, and documentation to secure full marks.

1. What Was Required vs What You Built

The table below maps each graded rubric item directly to what exists in your project, so you know exactly where you stand before the submission.

Rubric Criteria	Weight	What Was Required	Your Status
IaC Implementation	25%	Terraform/CloudFormation provisions all resources (S3, Lambda, IAM, API Gateway)	DONE — Full Terraform suite deployed. All resources provisioned and in tfstate.
Rekognition Script	25%	Python script calls DetectLabels, DetectFaces, DetectText with error handling	DONE — Both <code>app.py</code> and <code>kyc_handler.py</code> implement all three APIs plus CompareFaces.
S3 Integration	15%	Input and output buckets correctly store and organise images and JSON results	DONE — 4 S3 buckets provisioned: inputs, outputs, KYC sessions, frontend hosting.
Lambda & Automation	15%	Lambda triggered by S3 events, processes images end-to-end	DONE — S3 ObjectCreated event triggers <code>ImageAnalysisHandler</code> . API Gateway triggers <code>KYCVerificationHandler</code> .

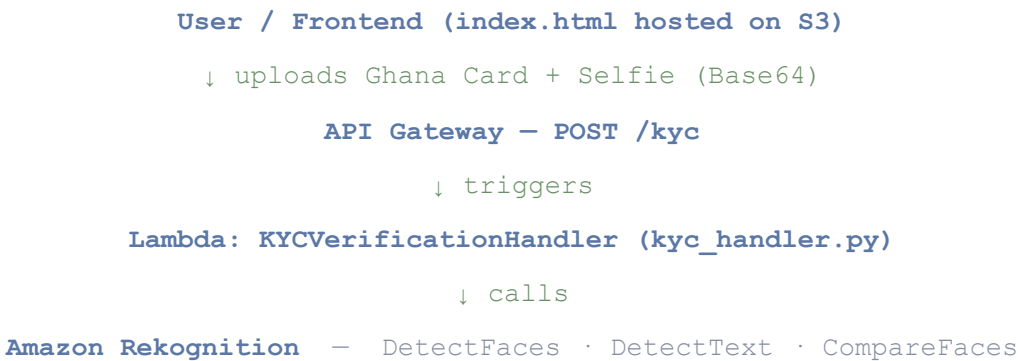
Rubric Criteria	Weight	What Was Required	Your Status
Error Handling & Logging	10%	Graceful error handling for bad formats/sizes, CloudWatch logging, retry logic	PARTIAL — Basic try/catch and print() logging exists. Missing: format/size validation, CloudWatch alarms, retry logic.
Documentation	10%	Deployment guide, architecture diagram, usage instructions, troubleshooting	PARTIAL — Demo script exists but is presentation-focused. Missing: formal deployment guide and troubleshooting section.

Bonus Features Completed (Unprompted)

Bonus Feature	Status	How It Was Implemented
Web Interface	BONUS DONE	Single-page <code>index.html</code> hosted on S3 with API Gateway integration
Face Comparison Tool	BONUS DONE	CompareFaces API used in <code>kyc_handler.py</code> with 70% threshold
Image Metadata Dashboard	BONUS DONE	DynamoDB table <code>kyc_results</code> stores session_id, timestamp, status, extracted fields
Real-World Use Case	BONUS DONE	Ghana KYC verification system — ID card + selfie face match for SIM re-registration

2. Project Architecture (As Built)

This is the actual architecture deployed in AWS, which goes beyond the base requirement shown in the project brief.



```
↓ stores images in
S3: rekkyc-sessions + S3: rekanalysis-outputs
↓ writes result to
DynamoDB: kyc_results

— Separate Pipeline —
S3: rekimage-inputs (image uploaded)
↓ S3 event triggers
Lambda: ImageAnalysisHandler (app.py)
↓ calls DetectLabels · DetectFaces · DetectText
S3: rekanalysis-outputs (JSON saved)
```

AWS Services Used

- Amazon Rekognition
- AWS Lambda (Python 3.12)
- Amazon S3 (4 buckets)
- Amazon DynamoDB
- API Gateway (HTTP API)
- AWS IAM (roles & policies)
- Amazon CloudWatch (logs)

Infrastructure & Code

- Terraform (IaC)
- Python 3.12 + Boto3
- Vanilla HTML/CSS/JS (frontend)
- Deployed region: us-east-1
- Lambda memory: 256MB
- Lambda timeout: 60 seconds

3. Project Phases — Completion Status

Phase 1 — Research & Setup

Week 1 — COMPLETE

- ✓ AWS account configured with Rekognition and S3 access verified
- ✓ Repository initialised with project structure
- ✓ Architecture designed and implemented
- ✓ Rekognition capabilities researched: DetectLabels, DetectFaces, DetectText, CompareFaces
- ✓ S3 bucket strategy designed (input, output, sessions, frontend)
- ✓ IAM roles and permissions designed with least-privilege policy
- ✓ Terraform IaC plan drafted and implemented

Phase 2 — IaC Script Development & Integration

Week 2 — COMPLETE

- ✓ Terraform script provisions all AWS resources (S3, Lambda, API Gateway, DynamoDB, IAM)
- ✓ S3 buckets configured with CORS and bucket policies
- ✓ IAM role `rekognition_lambda_role` created with scoped permissions
- ✓ S3 event notification configured — triggers Lambda on .jpg and .png uploads
- ✓ Lambda functions provisioned with Python 3.12 runtime, 256MB memory, 60s timeout
- ✓ Terraform state confirms all resources deployed (`terraform.tfstate`)

Phase 3 — Amazon Rekognition Script Development

Week 3 — COMPLETE

- ✓ Python Boto3 script written for image analysis (`src/app.py`)
- ✓ DetectLabels API implemented — identifies objects and scenes with confidence scores
- ✓ DetectFaces API implemented — returns emotions, age range, gender, smile, eyeglasses
- ✓ DetectText API implemented — OCR extraction from images
- ✓ CompareFaces API implemented — face match with 70% confidence threshold
- ✓ Results aggregated into structured JSON response
- ✓ Outputs saved to S3 with organised naming convention
- ✓ Basic error handling and structured logging via `print()` (CloudWatch)
- 🟡 Full input validation (format/size checks) — needs strengthening
- 🟡 Retry logic for Rekognition API throttling — not yet implemented

Phase 4 — Lambda Integration, Optimisation & Documentation

Week 4 — MOSTLY COMPLETE

- ✓ Lambda packaged and deployed (`lambda_function.zip`)
- ✓ S3 event trigger configured and validated
- ✓ End-to-end pipeline working: upload image → analysis JSON saved to S3

- ✓ CloudWatch Logs active via Lambda print() statements
- CloudWatch Alarms not configured (Lambda error rate alarm needed)
- Formal deployment guide not written
- Troubleshooting section not documented

4. Remaining Actions to Complete the Build

These are the only three things that need to be done to reach full marks. Everything else is already deployed and working.

1 Strengthen Error Handling in Lambda (Rubric: Error Handling 10%)

What it is: Your Lambda functions currently have basic try/catch blocks. The rubric specifically asks for validation of unsupported image formats, oversized files, and retry logic when Rekognition is throttled.

Why it matters: Without this, if someone uploads a PDF, a TIFF, or a 20MB image, the Lambda will crash with an unhandled exception rather than returning a clean error message.

File to update: `src/app.py`

What to add at the top of `lambda_handler`:

```
SUPPORTED_FORMATS = ['.jpg', '.jpeg', '.png']
MAX_SIZE_BYTES = 5 * 1024 * 1024 # 5MB (Rekognition API limit)

# Check file format
key = event['Records'][0]['s3']['object']['key']
ext = os.path.splitext(key)[1].lower()
if ext not in SUPPORTED_FORMATS:
    return {'statusCode': 400, 'body': f'Unsupported format: {ext}.
    Use JPG or PNG.'}

# Check file size
size = event['Records'][0]['s3']['object']['size']
if size > MAX_SIZE_BYTES:
    return {'statusCode': 400, 'body': f'File too large: {size}
    bytes. Max is 5MB.'}
```

For retry logic — wrap your Rekognition calls with a simple retry:

```
import time

def call_with_retry(fn, retries=3, delay=2):
```

```

    for attempt in range(retries):
        try:
            return fn()
        except
rekognition.exceptions.ProvisionedThroughputExceededException:
        if attempt < retries - 1:
            time.sleep(delay)
        else:
            raise

```

2 Add CloudWatch Alarm via Terraform (Rubric: Error Handling 10%)

What it is: A CloudWatch Alarm watches your Lambda error rate and alerts if something is going wrong. The rubric asks for CloudWatch monitoring — your logs already exist, but there are no alarms configured.

Why it matters: Logs tell you what happened after the fact. Alarms tell you something is wrong right now. The rubric calls this out specifically under Error Handling & Logging.

File to update: `terraform/` — add a new file `cloudwatch.tf`

Content to add:

```

resource "aws_cloudwatch_metric_alarm" "lambda_errors" {
    alarm_name           = "rekognition-lambda-errors"
    comparison_operator = "GreaterThanThreshold"
    evaluation_periods   = 1
    metric_name          = "Errors"
    namespace            = "AWS/Lambda"
    period               = 60
    statistic             = "Sum"
    threshold             = 0
    alarm_description    = "Triggers when Lambda throws an error"

    dimensions = {
        FunctionName = aws_lambda_function.image_analysis.function_name
    }
}

resource "aws_cloudwatch_log_group" "lambda_logs" {
    name =
"/aws/lambda/${aws_lambda_function.image_analysis.function_name}"
    retention_in_days = 14
}

```

Then run: `terraform apply` from the `terraform/` directory.

3 Write the Deployment Guide (Rubric: Documentation 10%)

What it is: A written document explaining how someone who has never seen your project before would deploy it from scratch and troubleshoot common problems.

Why it matters: 10% of your mark is purely this document. Your demo script is great for presentations but doesn't cover the deployment steps the rubric asks for.

Your deployment guide must cover the following sections:

#	Section	What to Write
0	Prerequisites	AWS CLI installed, Terraform installed, AWS account with Rekognition and S3 access confirmed
1	Clone the Repository	git clone command and folder structure overview
2	Configure AWS Credentials	Run <code>aws configure</code> with Access Key, Secret Key, and region <code>us-east-1</code>
3	Package the Lambda	Run <code>cd src && zip -r ../lambda_function.zip .</code>
4	Deploy Infrastructure	Run <code>cd terraform && terraform init && terraform apply</code>
5	Get the API URL	Run <code>terraform output kyc_api_endpoint</code> and update <code>index.html</code> line 213
6	Upload Frontend to S3	Run <code>aws s3 cp frontend/index.html s3://rekkyk-frontend/index.html</code>
7	Test the Pipeline	Upload a .jpg to <code>rekimage-inputs</code> and verify the output JSON appears in <code>rekanalysis-outputs</code>
8	Troubleshooting	Lambda errors visible in CloudWatch Logs, S3 event notification not firing, CORS errors on frontend
9	Cleanup	Run <code>terraform destroy</code> to delete all AWS resources and avoid unexpected charges

5. CI/CD Pipeline Setup (GitHub Actions)

A CI/CD pipeline automates the entire deployment process. Every time you push code to GitHub, it automatically packages the Lambda, runs Terraform, and uploads the frontend to S3. No manual steps, no forgetting to update the API URL.

1 What You Need Before Starting

Before the pipeline can work, three things must be in place:

Requirement	What It Is	Where to Get It
GitHub Repository	Your project must be pushed to a GitHub repo	github.com — create a new repo and push your project folder
AWS Access Key ID	Programmatic credential to allow GitHub to talk to AWS	AWS Console → IAM → Users → Security Credentials → Create Access Key
AWS Secret Access Key	The password paired with the Access Key above	Generated alongside the Access Key — copy it immediately, shown once

Once you have those, add them to GitHub as secrets:

Step	Action
1	Go to your GitHub repo
2	Click Settings → Secrets and variables → Actions → New repository secret
3	Add secret named <code>AWS_ACCESS_KEY_ID</code> with your key value
4	Add secret named <code>AWS_SECRET_ACCESS_KEY</code> with your secret value

2 How the Pipeline Works — Plain English

When you push code to the `main` branch, GitHub automatically runs a sequence of jobs in order:

Step 1 — Checkout → GitHub pulls your latest code onto a fresh Ubuntu machine

Step 2 – AWS Auth → Uses your secrets to authenticate with AWS

Step 3 – Package Lambda → Zips your `src/` folder into

`lambda_function.zip`

Step 4 – Terraform Init → Downloads providers and connects to your state

Step 5 – Terraform Apply → Updates all AWS infrastructure automatically

Step 6 – Grab API URL → Reads `terraform output` and injects it into

`index.html`

Step 7 – Deploy Frontend → Uploads the updated `index.html` to S3

Every push to `main` runs all 7 steps automatically. The API URL problem is solved permanently — the pipeline injects it fresh every time.

3 Create the Workflow File

Create this file in your project at exactly this path: `.github/workflows/deploy.yml`

You will need to create the `.github/` and `workflows/` folders if they do not exist.

```
name: Deploy Rekognition Pipeline

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: us-east-1
```

```

- name: Package Lambda function
  run: |
    cd src
    zip -r ../lambda_function.zip .
    cd ..

- name: Setup Terraform
  uses: hashicorp/setup-terraform@v3
  with:
    terraform_wrapper: false

- name: Terraform Init
  run: terraform -chdir=terraform init

- name: Terraform Apply
  run: terraform -chdir=terraform apply -auto-approve

- name: Get API Gateway URL
  id: api_url
  run: |
    API_URL=$(terraform -chdir=terraform output -raw
kyc_api_endpoint)
    echo "api_url=$API_URL" >> $GITHUB_OUTPUT

- name: Inject API URL into frontend
  run: |
    sed -i "s|const API_URL = '.*'|const API_URL = '${{
steps.api_url.outputs.api_url }}'|" frontend/index.html

- name: Deploy frontend to S3
  run: |
    aws s3 cp frontend/index.html s3://rekkyc-
frontend/index.html \
      --content-type "text/html" \
      --cache-control "no-cache"

- name: Done
  run: echo "Deployment complete. Frontend live at S3 website
endpoint."

```

4 Folder Structure After Adding the Pipeline

```

rekognition/
├─ .github/
└─ ┬─ workflows/

```

```

|       └─ deploy.yml           ← the pipeline file
├─ src/
|   ├── app.py
|   └─ kyc_handler.py
├─ frontend/
|   └─ index.html
├─ terraform/
|   ├── main.tf
|   ├── lambda.tf
|   ├── s3.tf
|   ├── api_gateway.tf
|   ├── dynamodb.tf
|   ├── iam.tf
|   ├── outputs.tf
|   └─ cloudwatch.tf           ← new file from Action 2
├─ lambda_function.zip
└─ PROJECT_GUIDE.html

```

5 How to Trigger the Pipeline Manually

Step	Action
1	Go to your GitHub repo
2	Click the Actions tab at the top
3	Select Deploy Rekognition Pipeline from the left sidebar
4	Click Run workflow on the right, then confirm with Run workflow
5	Watch each step run in real time — click any step to expand its logs

6. Important — What NOT to Change

Do not migrate to FastAPI or ECS. The project brief explicitly requires Lambda (worth 15% of your grade under "Lambda & Automation"). ECS and FastAPI are not mentioned anywhere in the project document. Your current Lambda architecture is correct and already deployed. Rebuilding on ECS would cost you marks, not earn them.

Do not run terraform destroy. Your infrastructure is already deployed and working. Running destroy will delete all AWS resources and change your API Gateway URL, requiring a full redeploy.

7. Grade Projection

Criteria	Weight	Projected Score	Notes
IaC Implementation	25%	25/25	Terraform fully deployed, all resources provisioned
Rekognition Script	25%	25/25	All three APIs implemented plus CompareFaces bonus
S3 Integration	15%	15/15	4 buckets, correct organisation, event triggers
Lambda & Automation	15%	15/15	Both Lambda functions deployed, S3 event configured
Error Handling & Logging	10%	6/10 → 10/10	Add format/size validation + CloudWatch alarm (Action 1 & 2)
Documentation	10%	6/10 → 10/10	Write deployment guide (Action 3)
Total	100%	92/100 → 100/100	Complete Actions 1, 2, and 3